

SI PROJECT DEFENSE

FABRIC DEFECT DETCTION WITH COMPUTER VISION

PRESENTED BY:

Shristi

Shruthi

Harshitha

Jayamadhuri

Abinash

TEAMMATES



Harshitha Borra



Shruthi



Shristi



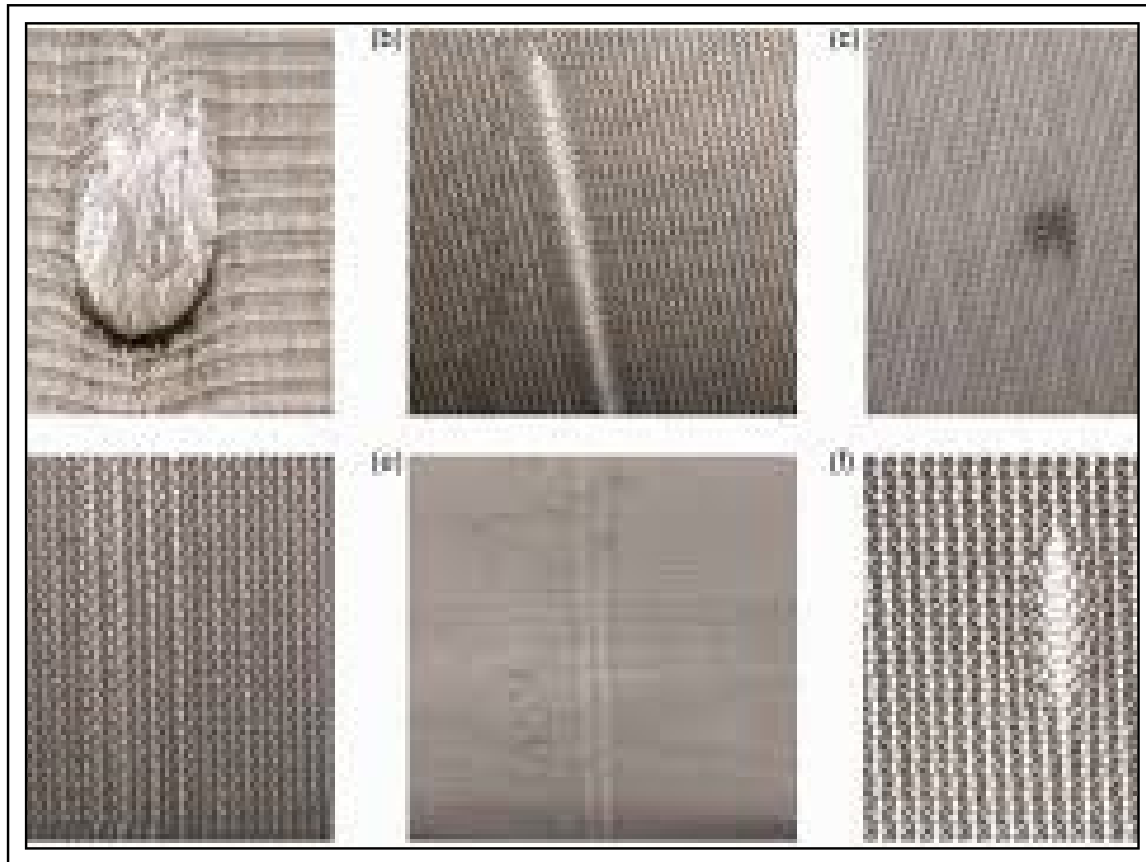
Jayamadhuri



Abinash A

INDEX

01	INTRODUCTION
02	LITERATURE REVIEW
03	DATA COLLECTION AND PREPERATION
04	YOLO MODEL
05	RESULT
06	DEFECT DETECTION WEBSITE
07	SWOT
08	CONCLUSION



FABRIC DEFECT DETECTION

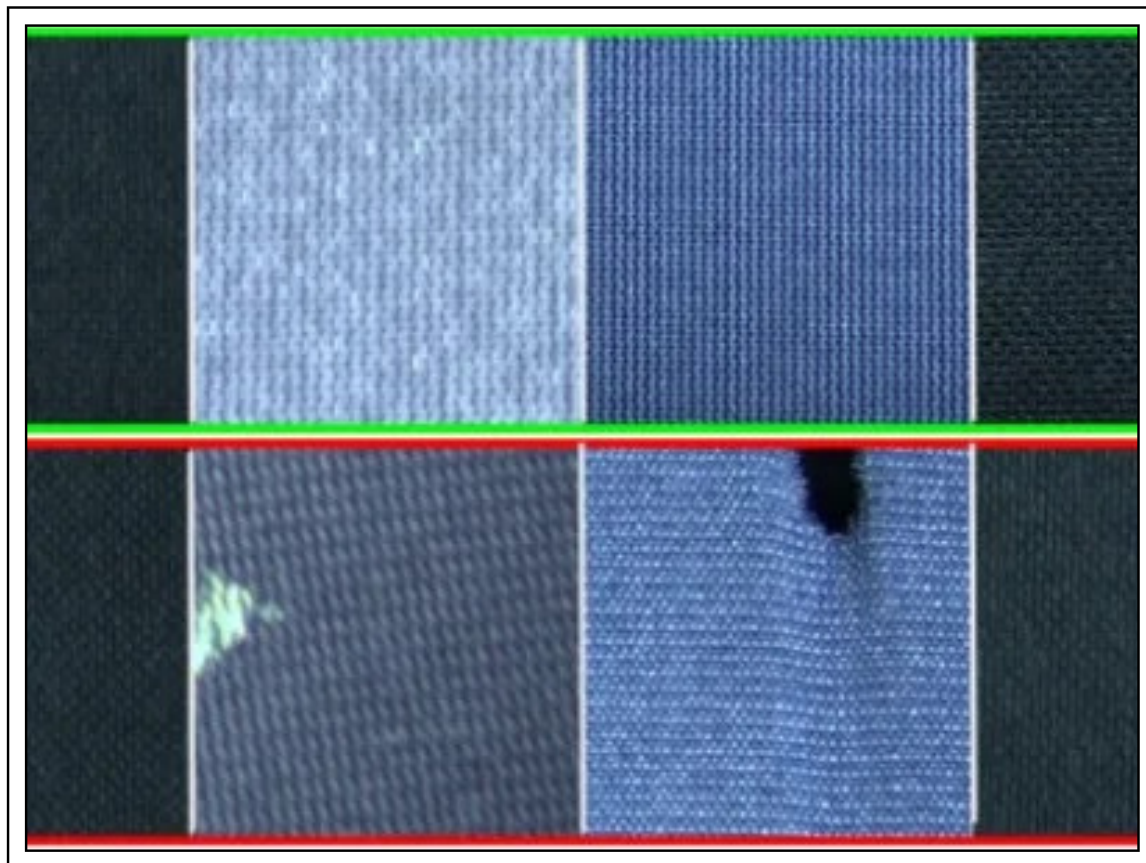
Fabric quality control is a critical step in textile manufacturing because it ensures product quality and reduces production errors. However, traditional manual inspection is often slow, labor-intensive, and subjective.

Problem Statement

Manual inspection is slow, subjective, and inefficient

Aim

Build an image-based system to classify fabric as OK/NG and localize defects.

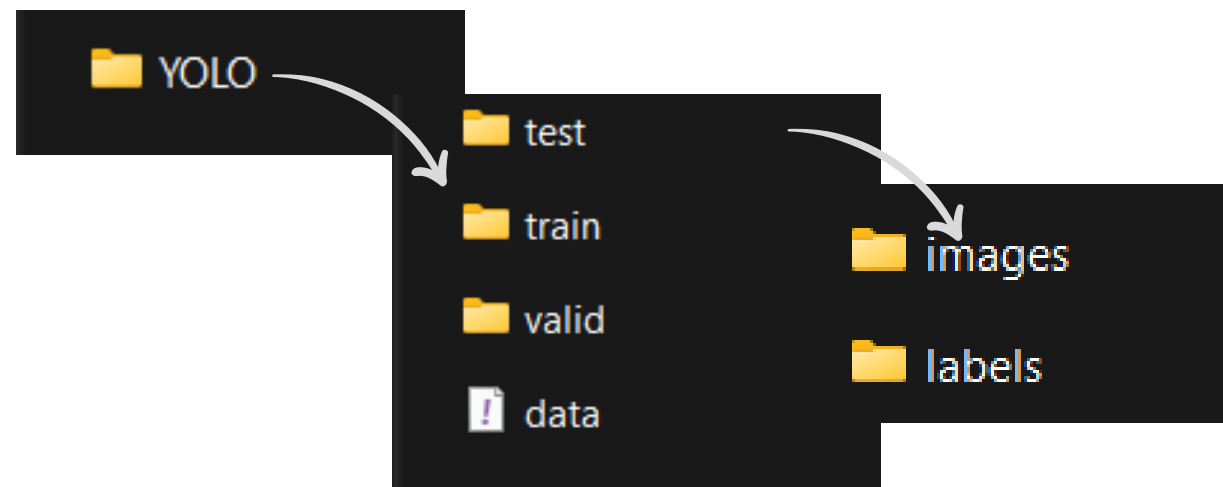


LITERATURE REVIEW

Method	Key Idea	Advantage	Limitation
CNNs	Learn features from images automatically	High accuracy	Requires large labeled datasets
Autoencoders	Reconstruct normal images and detect anomalies	Works with fewer defect samples	May miss subtle defectss
YOLO	Detects and localizes defects using bounding boxes	Fast and real-time	Needs labeled defect data
PatchCore	Uses memory bank of normal features	Strong anomaly detection	High memory usage
PaDiM	Models normal patch distributions	Accurate localization	Computationally expensive

DATA COLLECTION

DATASET PREPARATION

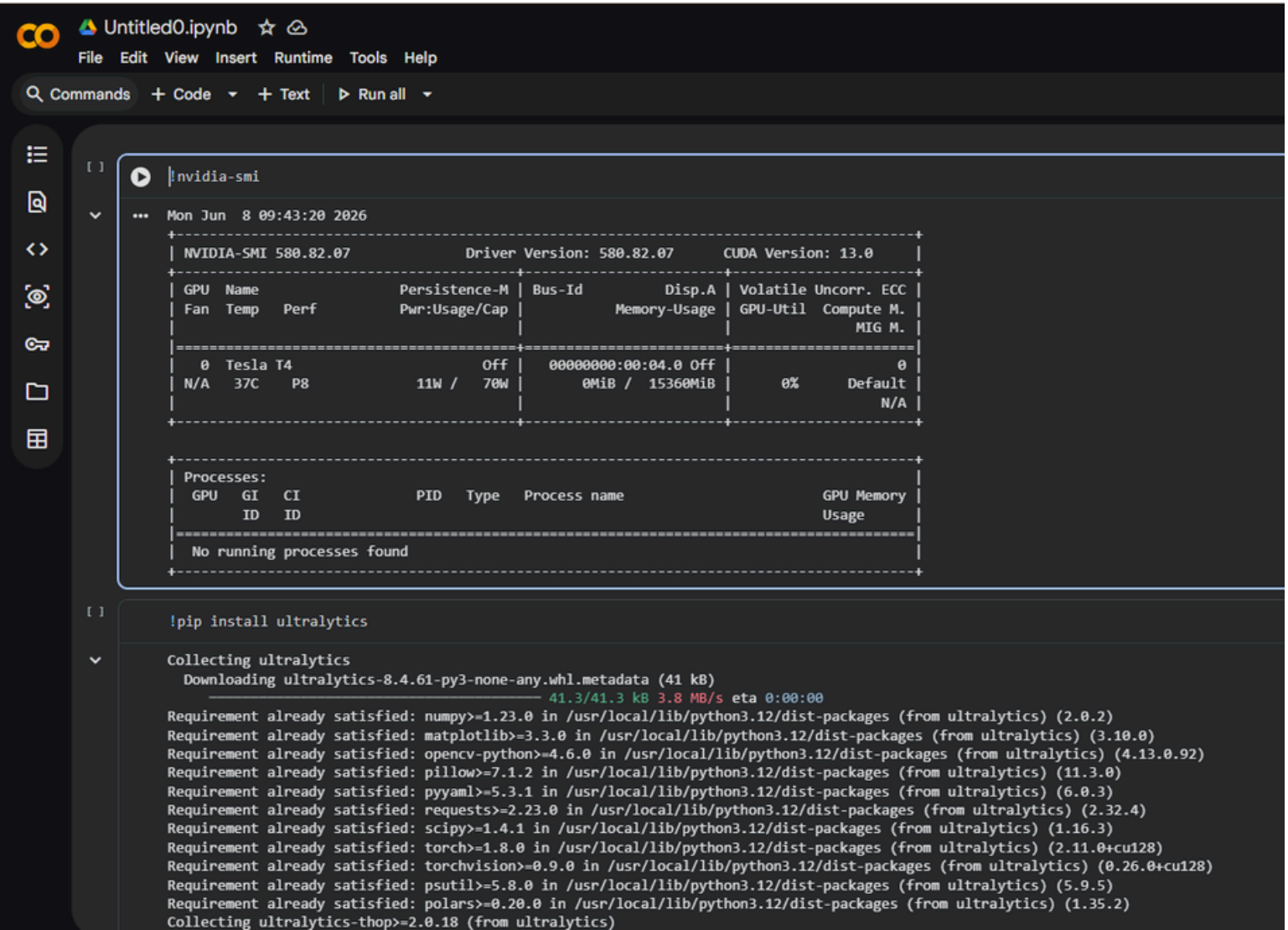


Dataset	Images
Training	540
Validation	73
Testing	107
Total	720

- Fabric images with and without defects collected
- Defects included:
 - Holes
 - Stains
 - Broken yarns
 - Weaving irregularities
- The training dataset was used to teach the model the visual characteristics of fabric defects.
- The validation dataset was used during training to monitor performance and prevent overfitting.
- Testing dataset was reserved for evaluating the final model on unseen images.

DEVELOPMENT ENVIRONMENT

- Google Colab was used as the cloud-based platform for developing and training the fabric defect detection system.
- NVIDIA T4 GPU was enabled in Google Colab to accelerate deep learning model training.
- Python was used for implementation due to its strong support for machine learning and computer vision.
- YOLOv8 (Ultralytics) was used for fabric defect detection, providing tools for training, testing, and deployment.



Tool	Purpose
Python	Programming Language
YOLOv8	Object Detection Framework
T4 GPU	Accelerated Training
OpenCV	Image Processing
Matplotlib	Result Visualization

MODEL TRAINING DEVELOPMENT

- YOLOv8 Nano (YOLOv8n) was selected due to its lightweight architecture, fast processing speed, and good detection accuracy.
- Transfer Learning was applied using the pre-trained model to leverage previously learned visual features.
- Model Training was performed on the uploaded fabric defect dataset using a 640×640 image size and the dataset configuration file .
- 50 Epochs were used during training, allowing the model to repeatedly learn defect patterns and improve detection performance.

```
import os

from ultralytics import YOLO

model = YOLO("yolov8n.pt")

model.train(
    data="/content/FD_Dataset/YOLO/data.yaml",
    epochs=50,
    imgsz=640
)
```

...	0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
	1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
	2	-1	1	7360	ultralytics.nn.modules.block.C2f	[32, 32, 1, True]
	3	-1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
	4	-1	2	49664	ultralytics.nn.modules.block.C2f	[64, 64, 2, True]
	5	-1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
	6	-1	2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]
	7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
	8	-1	1	460288	ultralytics.nn.modules.block.C2f	[256, 256, 1, True]
	9	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
	10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
	11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	12	-1	1	148224	ultralytics.nn.modules.block.C2f	[384, 128, 1]
	13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
	14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	15	-1	1	37248	ultralytics.nn.modules.block.C2f	[192, 64, 1]
	16	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
	17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	18	-1	1	123648	ultralytics.nn.modules.block.C2f	[192, 128, 1]
	19	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
	20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	21	-1	1	493056	ultralytics.nn.modules.block.C2f	[384, 256, 1]
	22	[15, 18, 21]	1	752092	ultralytics.nn.modules.head.Detect	[4, 16, None, [64, 128, 256]]

Model summary: 130 layers, 3,011,628 parameters, 3,011,612 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'

Parameter	Value
Model	YOLOv8n
Epochs	50
Image Size	640 × 640
Framework	Ultralytics YOLOv8

EPOCHS TRAINING PROCESS

- An epoch represents one complete pass of the model through the entire training dataset.
- For a dataset of 100 images, each epoch allows the model to analyze all 100 images once before starting the next cycle.
- The model was trained for 50 epochs, meaning the complete dataset was processed fifty times to improve learning.
- Training performance was monitored using metrics such as classification loss, bounding box loss, and mean Average Precision (mAP).

```
      -1  1    123648 ultralytics.nn.modules.block.C2f
      -1  1    147712 ultralytics.nn.modules.conv.Conv
    [-1, 9] 1         0 ultralytics.nn.modules.conv.Concat
      -1  1    493056 ultralytics.nn.modules.block.C2f
    [15, 18, 21] 1    752092 ultralytics.nn.modules.head.Detect
summary: 130 layers, 3,011,628 parameters, 3,011,612 gradients, 8.2 GFLOPs

ferred 319/355 items from pretrained weights
ing layer 'model.22.dfl.conv.weight'
running Automatic Mixed Precision (AMP) checks...
loading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26
checks passed ✓
: Fast image access ✓ (ping: 0.0±0.0 ms, read: 1758.9±970.6 MB/s, size: 69.
: Scanning /content/FD_Dataset/YOLO/train/labels... 540 images, 0 background
: New cache created: /content/FD_Dataset/YOLO/train/labels.cache
entations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3
Fast image access ✓ (ping: 0.0±0.0 ms, read: 407.2±64.5 MB/s, size: 76.8 KB
Scanning /content/FD_Dataset/YOLO/valid/labels... 73 images, 0 backgrounds,
New cache created: /content/FD_Dataset/YOLO/valid/labels.cache
izer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and d
izer: AdamW(lr=0.00125, momentum=0.9) with parameter groups 57 weight(decay=
ing labels to /content/runs/detect/train/labels.jpg...
  sizes 640 train, 640 val
  2 dataloader workers
ng results to /content/runs/detect/train
ing training for 50 epochs...

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
 1/50   1.99G    2.059    4.365    2.005         12    640: 10
      Class  Images  Instances  Box(P      R      mAP50  mA
      all      73         81    0.00492    0.664    0.109

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
 2/50   2.46G    1.987    3.643    1.908         16    640: 79
```

YOU

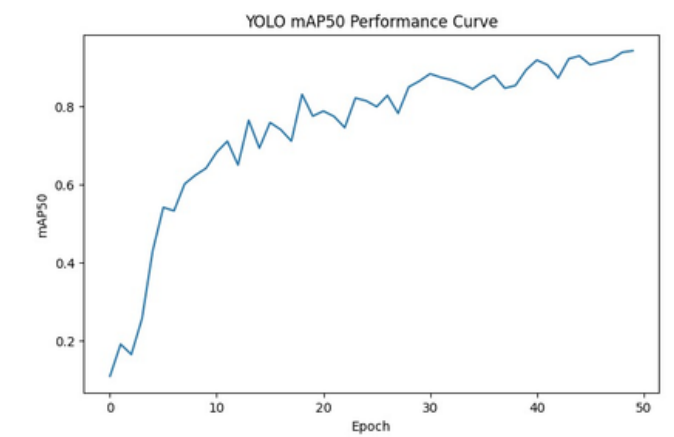
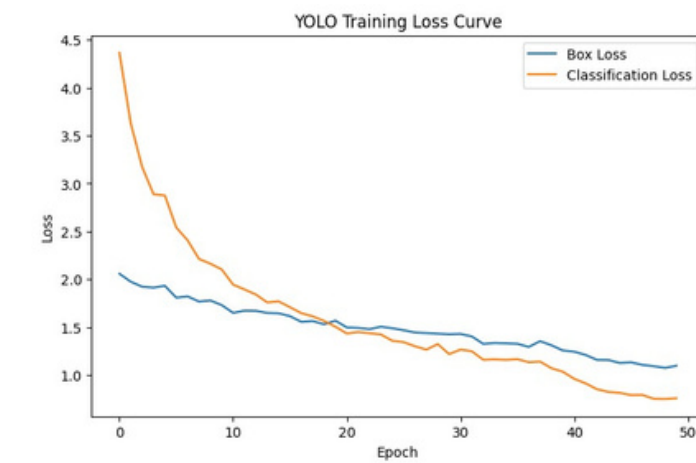
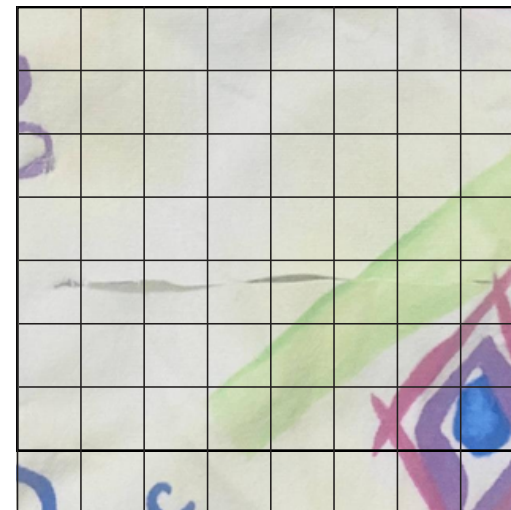
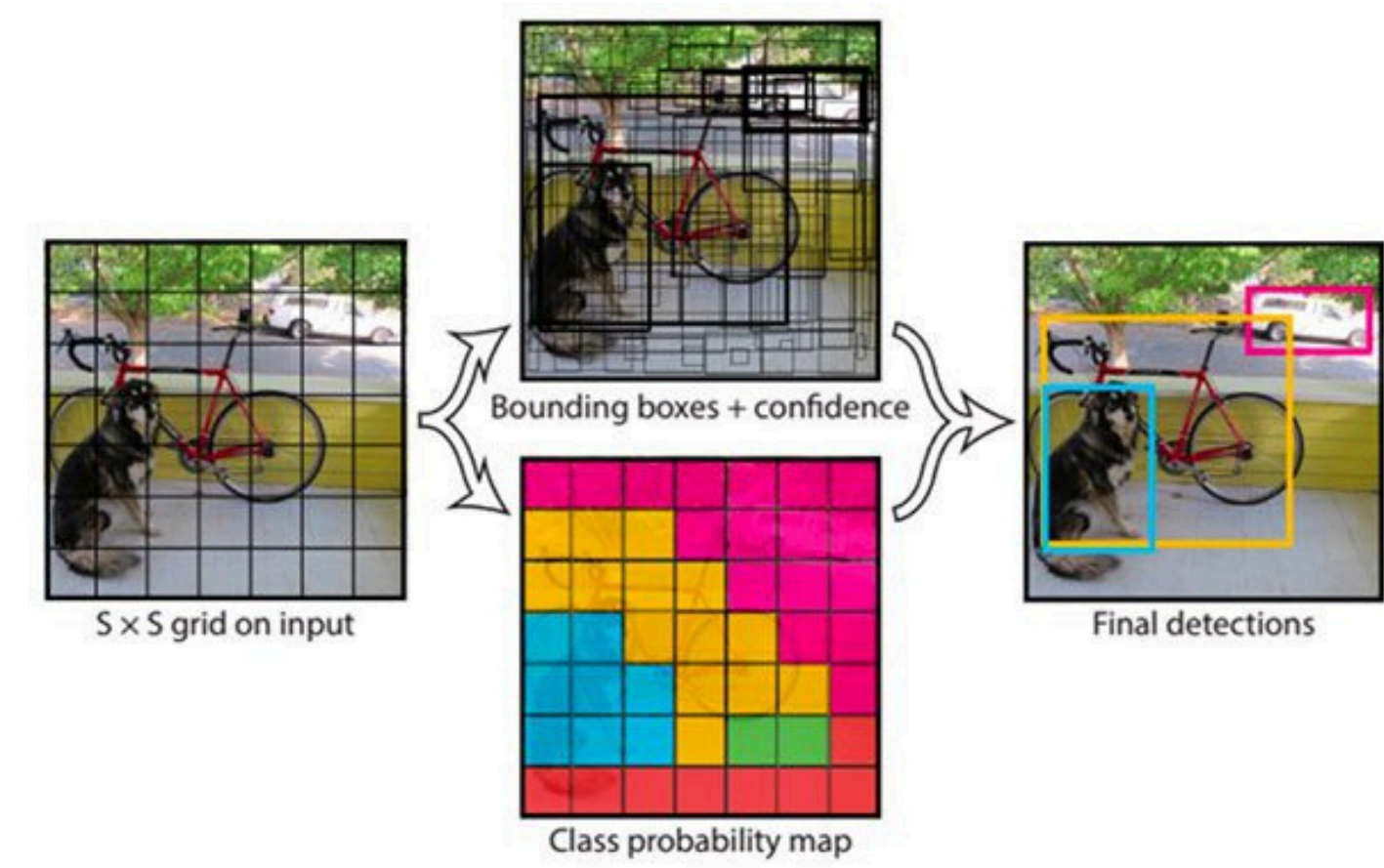
ONLY

LOOK

ONCE

YOLO

YOLOv8 Nano





Google Collab



```
!pip install ultralytics
```

```
Collecting ultralytics
  Downloading ultralytics-8.4.61-py3-none-any.whl.metadata (41 kB)
    41.3/41.3 kB 3.8 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0.92)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: torchvision>=0.17.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.16.3)
```

```
from ultralytics import YOLO
```

```
*** Creating new Ultralytics Settings v0.0.6 file [x]
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=~/path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving FD_Dataset.zip to FD_Dataset.zip

Files

Step 2: Create the app.py file

```
This magic command writes the Python code directly into a file named app.py in your Colab environment. Note that I am using the path to your trained model: /content/runs/detect/train-3/weights/best.pt.

11 # Writefile app.py
import streamlit as st
from ultralytics import YOLO
from PIL import Image
import tempfile
import os

# Load the model you just trained
model_path = "/content/runs/detect/train-3/weights/best.pt"
if os.path.exists(model_path):
    model = YOLO(model_path)
else:
    st.error("Model file not found. Please check the path.")

st.title("AI Fabric Quality Inspection System")
st.write("Upload a fabric image to detect defects.")

uploaded_file = st.file_uploader("Choose an image...", type=["jpg", "jpeg", "png"])

if uploaded_file is not None:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Image", use_container_width=True)

    # Temporary save for YOLO
    with tempfile.NamedTemporaryFile(delete=False, suffix=".jpg") as tmp:
        image.save(tmp.name)
        results = model(tmp.name)

    # Show results
    annotated_image = results[0].plot()
    st.subheader("Detection Result")
    st.image(annotated_image, use_container_width=True)

    if len(results[0].boxes) > 0:
        st.error(f"Status: ❌ Defects found! ({len(results[0].boxes)} defects found)")
        for box in results[0].boxes:
            st.write(f"Model names: {list(box.cls[0])} (Conf: {float(box.conf[0]).:.2f})")
    else:
        st.success("Status: ✅ OK")
```

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

41/50 2.466 1.246 0.9624 1.416 12 640 100% 34/34 3.51t/s 9.9s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 1/3 2.71t/s 1.1s

all 73 81 0.96 0.83 0.919 0.511

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

42/50 2.466 1.212 0.9152 1.364 12 640 100% 34/34 4.61t/s 7.4s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.9s

all 73 81 0.923 0.798 0.907 0.517

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

43/50 2.466 1.16 0.8542 1.34 12 640 100% 34/34 4.11t/s 8.2s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.8s

all 73 81 0.924 0.802 0.873 0.517

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

44/50 2.466 1.159 0.8257 1.353 15 640 100% 34/34 3.91t/s 8.7s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.41t/s 0.9s

all 73 81 0.902 0.864 0.923 0.514

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

45/50 2.466 1.150 0.8163 1.338 12 640 100% 34/34 4.61t/s 7.3s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 2.91t/s 1.0s

all 73 81 0.863 0.915 0.93 0.514

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

46/50 2.466 1.114 0.7925 1.319 12 640 100% 34/34 4.31t/s 7.8s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.9s

all 73 81 0.898 0.894 0.907 0.55

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

47/50 2.466 1.100 0.7941 1.306 12 640 100% 34/34 4.21t/s 8.1s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.71t/s 0.8s

all 73 81 0.874 0.882 0.915 0.54

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

48/50 2.466 1.094 0.7539 1.275 12 640 100% 34/34 4.21t/s 8.1s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 2.31t/s 1.3s

all 73 81 0.896 0.872 0.921 0.542

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

49/50 2.466 1.077 0.7522 1.298 13 640 100% 34/34 4.61t/s 7.3s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.61t/s 0.8s

all 73 81 0.895 0.919 0.939 0.55

What can I help you build?

Gemini 3 Flash

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

41/50 2.466 1.246 0.9624 1.416 12 640 100% 34/34 3.51t/s 9.9s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 1/3 2.71t/s 1.1s

all 73 81 0.96 0.83 0.919 0.511

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

42/50 2.466 1.212 0.9152 1.364 12 640 100% 34/34 4.61t/s 7.4s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.9s

all 73 81 0.923 0.798 0.907 0.517

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

43/50 2.466 1.16 0.8542 1.34 12 640 100% 34/34 4.11t/s 8.2s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.8s

all 73 81 0.924 0.802 0.873 0.517

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

44/50 2.466 1.159 0.8257 1.353 15 640 100% 34/34 3.91t/s 8.7s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.41t/s 0.9s

all 73 81 0.902 0.864 0.923 0.514

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

45/50 2.466 1.150 0.8163 1.338 12 640 100% 34/34 4.61t/s 7.3s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 2.91t/s 1.0s

all 73 81 0.863 0.915 0.93 0.514

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

46/50 2.466 1.114 0.7925 1.319 12 640 100% 34/34 4.31t/s 7.8s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.9s

all 73 81 0.898 0.894 0.907 0.55

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

47/50 2.466 1.100 0.7941 1.306 12 640 100% 34/34 4.21t/s 8.1s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.71t/s 0.8s

all 73 81 0.874 0.882 0.915 0.54

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

48/50 2.466 1.094 0.7539 1.275 12 640 100% 34/34 4.21t/s 8.1s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 2.31t/s 1.3s

all 73 81 0.896 0.872 0.921 0.542

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

49/50 2.466 1.077 0.7522 1.298 13 640 100% 34/34 4.61t/s 7.3s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.61t/s 0.8s

all 73 81 0.895 0.919 0.939 0.55

What can I help you build?

Gemini 3 Flash

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

41/50 2.466 1.246 0.9624 1.416 12 640 100% 34/34 3.51t/s 9.9s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 1/3 2.71t/s 1.1s

all 73 81 0.96 0.83 0.919 0.511

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

42/50 2.466 1.212 0.9152 1.364 12 640 100% 34/34 4.61t/s 7.4s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.9s

all 73 81 0.923 0.798 0.907 0.517

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

43/50 2.466 1.16 0.8542 1.34 12 640 100% 34/34 4.11t/s 8.2s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.8s

all 73 81 0.924 0.802 0.873 0.517

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

44/50 2.466 1.159 0.8257 1.353 15 640 100% 34/34 3.91t/s 8.7s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.41t/s 0.9s

all 73 81 0.902 0.864 0.923 0.514

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

45/50 2.466 1.150 0.8163 1.338 12 640 100% 34/34 4.61t/s 7.3s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 2.91t/s 1.0s

all 73 81 0.863 0.915 0.93 0.514

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

46/50 2.466 1.114 0.7925 1.319 12 640 100% 34/34 4.31t/s 7.8s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.51t/s 0.9s

all 73 81 0.898 0.894 0.907 0.55

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

47/50 2.466 1.100 0.7941 1.306 12 640 100% 34/34 4.21t/s 8.1s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.71t/s 0.8s

all 73 81 0.874 0.882 0.915 0.54

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

48/50 2.466 1.094 0.7539 1.275 12 640 100% 34/34 4.21t/s 8.1s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 2.31t/s 1.3s

all 73 81 0.896 0.872 0.921 0.542

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size

49/50 2.466 1.077 0.7522 1.298 13 640 100% 34/34 4.61t/s 7.3s

Class Images Instances Box(P R mAP50 mAP50-95) 100% 3/3 3.61t/s 0.8s

all 73 81 0.895 0.919 0.939 0.55

What can I help you build?

Gemini 3 Flash

QUALITY CLASSIFICATION

FABRIC STATUS : NG (DEFECT DETECTED)

Defect Report

Total Defects Found

1

Average Confidence

40.8%

	Defect Type	Confidence (%)
0	Broken_Yarn	40.8

FABRIC STATUS : GOOD (NO DEFECT FOUND)

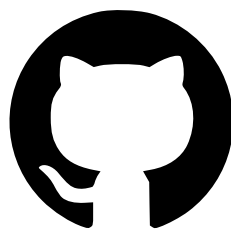
Detected Defects

0

Developed using YOLOv8 and Streamlit for Fabric Quality Inspection



FABRIC DEFECT DETECTION



PROJECT RESULT

- THE YOLO MODEL WAS SUCCESSFULLY TRAINED FOR FABRIC DEFECT DETECTION.
- THE MODEL DETECTED DEFECT REGIONS AND DISPLAYED THEM USING BOUNDING BOXES.
- FABRIC IMAGES WERE CLASSIFIED BASED ON THE PRESENCE OF DETECTED DEFECTS.
- GOOGLE COLAB WITH TESLA T4 GPU ENABLED EFFICIENT MODEL TRAINING AND TESTING.
- THE SYSTEM PROVIDED QUICK AND CONSISTENT INSPECTION RESULTS.

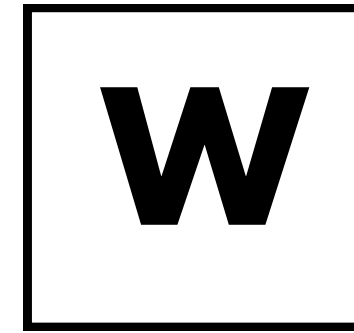
PROJECT DISCUSSION

- THE MODEL DEMONSTRATED THE ABILITY TO AUTOMATICALLY IDENTIFY FABRIC DEFECTS, REDUCING THE NEED FOR MANUAL INSPECTION.
- DEFECT LOCALIZATION HELPS INSPECTORS EASILY IDENTIFY PROBLEMATIC AREAS IN THE FABRIC.
- THE AUTOMATED APPROACH IMPROVES INSPECTION SPEED AND CONSISTENCY COMPARED TO TRADITIONAL VISUAL INSPECTION METHODS.
- THE RESULTS INDICATE THAT AI-BASED DEFECT DETECTION CAN SUPPORT TEXTILE QUALITY CONTROL PROCESSES AND REDUCE HUMAN ERROR.

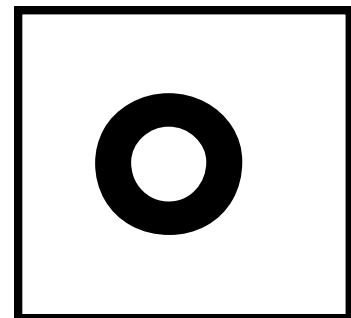
SWOT ANALYSIS



- Fast real-time detection
- Reduces manual inspection effort



- Requires labeled training data
- Small defects may be missed



- Industrial automation
- Smart textile manufacturing



- Insufficient defect image data for training
- Hardware and deployment costs

PROJECT CONCLUSION

- ✓ DEVELOPED YOLOV8-BASED FABRIC DEFECT DETECTION SYSTEM
- ✓ SUCCESSFULLY DETECTED AND LOCALIZED FABRIC DEFECTS
- ✓ CLASSIFIED FABRICS AS:
 - OK
 - MINOR DEFECT – ACCEPTABLE
 - NG
- ✓ FASTER AND MORE CONSISTENT THAN MANUAL INSPECTION
- ✓ POTENTIAL FOR REAL-TIME INDUSTRIAL IMPLEMENTATION

Resource Page

- YOLO MODEL
- [HTTPS://DEFECTDETECTIONINFABRIC.STREAMLIT.A
PP/](https://defectdetectioninfabric.streamlit.app/)

THANK YOU